

QUANTUM SEED GENERATOR v1.5.1 — System Documentation

Hardware-entropy based BIP39 seed generator on Raspberry Pi 5. As of: 30 August 2026 (script version 1.5.1, after four external audit rounds). This documentation describes purpose, design, cryptography and the security architecture of the system.

1. Purpose and Functional Description

The QUANTUM SEED GENERATOR produces BIP39 seed phrases (12 or 24 words) for Bitcoin wallets from four mutually independent physical entropy sources. The system runs fully offline on a Raspberry Pi 5, processes all secrets exclusively in RAM and leaves no traces on storage media. The resulting word sequence is standards-compliant and can be imported into any BIP39-compatible wallet — hardware wallets such as ColdCard, Trezor, Ledger, OneKey, BitBox as well as software wallets such as Electrum, Sparrow, Trust Wallet, BlueWallet and many more: the standard guarantees that every compatible wallet derives the same keys.

- Four entropy sources: radioactive decay, IR sensor noise, hardware RNG, dice
- Bilingual guidance (German/English) with standard and transparency mode
- Full auditability: transparency mode shows all raw data and hashes live
- Enforced offline environment: WLAN/Bluetooth shutdown (rfkill + firmware), Ethernet cable check
- Hardware health check, statistical decay test, paper verification of the transcript
- Trace-free finish: screen/scrollback erase, optional shell-history cleanup

2. Strengths versus Seed Generation in Standard Hardware Wallets

Commercial hardware wallets derive the seed from a single, proprietary on-chip random number generator whose inner workings the user can neither inspect nor verify. This system takes the opposite approach:

Aspect	Hardware wallet	This system
Entropy sources	1 (chip RNG, black box)	4 independent ones, incl. 1 quantum-physical (decay) and 1 hardware-independent (dice)
Auditability	none (closed source/silicon)	complete: open-source script, transparency mode shows every intermediate value
RNG backdoor risk	total loss of security	neutralised: HMAC combiner with dice as independent key
Entropy evidence	vendor claim	conservatively accounted and enforced (min. 2× target)
Supply-chain attack	device could arrive manipulated	commodity parts, self-assembled, behaviour measurable
Seed display environment	device display	enforced offline re-check immediately before display (tamper detection)

Important context: a high-quality hardware wallet remains sensible for storage and signing. This system only replaces the seed GENERATION step with a transparent multi-source process — the finished seed can afterwards be imported into a hardware wallet.

3. Technical Components

Component	Type / model	Role
Single-board computer	Raspberry Pi 5 (BCM2712), Raspberry Pi OS	execution, data acquisition, cryptography
Geiger counter	RadiationD-v1.1 (CAJOE) with J315 tube	quantum entropy: timestamps of radioactive decays (background radiation)
Thermal sensor	MLX90640 (32×24 thermopile, I2C 0x33)	sensor noise: uncalibrated raw ADC frames
Hardware RNG	BCM2712 TRNG (/dev/hwrng)	5 × 512 bits as separate samples with thermal excitation in between; quality checks per sample
Dice	precision dice (casino grade recommended)	hardware-independent final source, HMAC key
Resistor	10 kΩ (CAJOE signal line)	current limiting / GPIO input protection
Software	btc_seedgen.py + install_seedgen.py (Python 3, lgpio, smbus2)	acquisition, checks, combining, BIP39 encoding

4. Technical Wiring

All references use the Raspberry Pi 5 40-pin header (physical pin numbers; orientation: USB/Ethernet facing down, pin 1 top left, corner next to the SD card). The complete map of all 40 pins is available from the installer: `python3 install_seedgen.py --pinout`

MLX90640 IR camera

Camera wire	Pi pin	Function
VCC (yellow)	Pin 1	3.3 V — never 5 V!
SDA (violet)	Pin 3	GPIO2 / I2C1 SDA
SCL (green)	Pin 5	GPIO3 / I2C1 SCL
GND (blue)	Pin 6	ground

CAJOE Geiger counter

Connection	Pi pin	Note
5 V supply	Pin 2 (or its own USB PSU)	board power
GND	Pin 9	common ground is mandatory
Signal (white)	Pin 11 (GPIO17)	tapped at P3 VIN, 10 k Ω in series; crucial: GPIO17 bias-free (pull none) — set permanently by the installer (gpio=17=ip,pn)

5. Cryptography of the Seed Generation Process

5.1 Abstract explanation

Think of the process as mixing an unforgeable paint colour: four people who do not know each other each pour a private, secret tint into a blender (decay timing, camera noise, chip randomness, dice). The blender (hash function) stirs so thoroughly that none of the original tints can be reconstructed from the mix — and a single different molecule would yield a completely different result. Crucially, even if three of the four collude, the mix stays unpredictable as long as the fourth is honest. The dice play a special role: they are the 'key to the blender' — a source no chip maker, firmware or supplier could ever influence, because it comes from the user's own hand.

5.2 Technical explanation — step by step

Step 1 — raw data acquisition. Every source contributes complete raw data: the HWRNG 5×64 bytes as separate samples (with thermal CPU excitation between samples); the Geiger counter the kernel timestamps of all decay events in nanoseconds (8 bytes each, interrupt-accurate); the camera complete uncalibrated RAM frames (832 16-bit words = 1664 bytes per frame, only frames with $\geq 85\%$ pixel change versus the last accepted frame); the dice the full roll sequence encoded as a base-6 integer.

Step 2 — domain-separated hashing per source. Each source is condensed individually through SHA-512. A domain prefix and a length field prevent data of one source from being interpretable as another (format unambiguity):

```
H_x  = SHA-512( "BTC-SEEDGEN-v1" || tag_x || len64(daten_x) || daten_x )
tags : hwrng, decay, cam, dice, os
```

Step 3 — hardware pool. The three hardware digests are hashed again together with an additional operating-system contribution (os.urandom(64), defence in depth):

```
H_os  = SHA-512( DOM || "os"      || len || os.urandom(64) )
POOL  = SHA-512( DOM || "pool"    || H_hw || H_dec || H_cam || H_os )
```

Step 4 — HMAC combination with the dice as key. The core of the scheme: HMAC-SHA512 is a provably robust two-source combiner. Its output is unpredictable as long as AT LEAST ONE side (key or message) is unpredictable. Thus the dice rescue the seed even under fully compromised hardware — and vice versa:

```
FINAL = HMAC-SHA512( key = H_dice , msg = POOL )          # 512 Bit
```

Step 5 — entropy extraction and BIP39 encoding. For 24 words the first 256 bits of FINAL are taken (a prefix of a PRF output is itself a PRF output — losslessly secure, same principle as SHA-512/256). The BIP39 checksum is appended (first ENT/32 = 8 bits of SHA-256(ENTROPY)); the 264 bits are split into 24 groups of 11 bits, each indexing a word of the official wordlist (2048 words, verified via SHA-256).

Step 6 — wallet side (outside this system). On import the wallet performs the standard PBKDF2-HMAC-SHA512 with 2048 iterations (mnemonic + optional passphrase) and derives keys via BIP32/BIP84. Iterations inside the generator would be security theatre: key stretching only helps weak secrets — 256 bits of true entropy need no stretching.

5.3 The entropy sources and their contribution to overall security

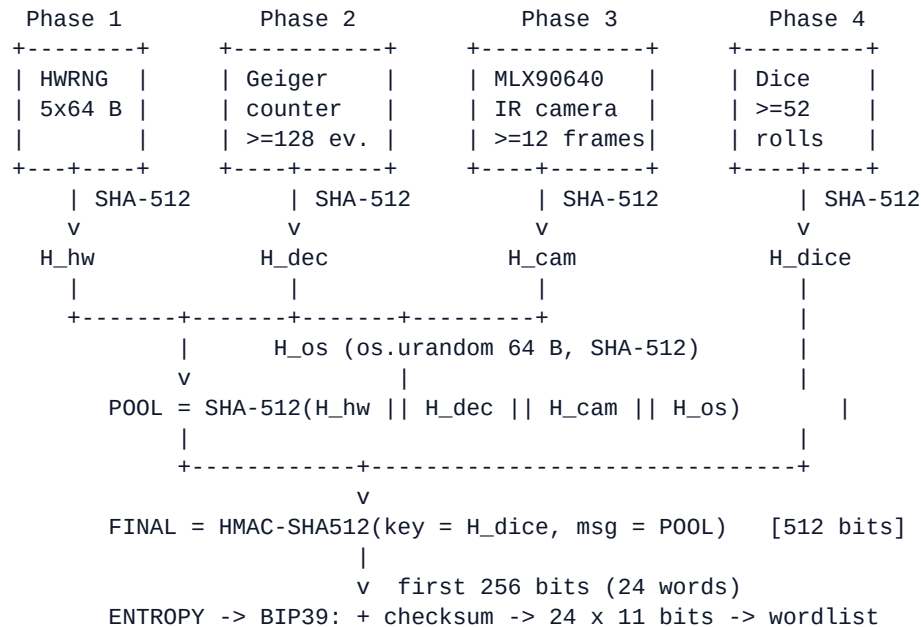
Source	Physical origin	Conservative crediting	Distinct value
Radioactive decay	quantum mechanics: decay instant fundamentally unpredictable	2 bits/event, min. 128 events (= 256 bits)	only source with fundamental (not merely practical) unpredictability; Poisson statistics verifiable
IR sensor noise	thermal noise + ADC quantisation across 768 pixels	64 bits/frame, min. 12 frames; 85 % filter enforces freshness	high data rate; user hand movement couples in macroscopic unpredictability
BCM2712 HWRNG	on-chip TRNG	512 bits raw, 256 credited; monobit/diversity checks	established baseline; as ONE of four sources uncritical even if defective
Dice	chaotic mechanics in the user's hand	$\log_2(6) \approx 2.585$ bits/roll, min. 52 rolls (≈ 134 bits), 100 recommended (≈ 258 bits)	independent of hardware and supply chain; HMAC key = last line of defence

Combiner security statement: the total seed entropy is $\min(\text{input entropy}, 256 \text{ bits})$, and an attacker would have to predict ALL four sources simultaneously. The balance check additionally enforces that the conservatively credited sum is at least twice the target.

5.4 Detailed Process Walkthrough with Illustrations and Worked Examples

This section walks through every phase individually — with data-flow diagrams, concrete byte layouts and verifiable numeric examples. All example values were computed by machine with the same algorithms the script itself uses.

5.4.1 Overall data flow



Reading: the four sources pass through domain-separated hashes individually. Three hardware sources plus the OS contribution form the pool (message); the dice deliberately stay outside and enter only as the HMAC key — hence ONE honest side suffices for a secure seed.

5.4.2 Phase 1 — HWRNG: 5 samples, thermal excitation, monobit example

Five samples of 64 bytes each are read from `/dev/hwrng` — spaced 2 seconds apart. During each pause a time-boxed SHA-512 burst heats the CPU: the next sample is taken under changed physical conditions (die temperature, supply load) — the jitter of the TRNG ring oscillators depends on temperature and voltage. This is decorrelation reinforcement by the same principle as the hand movement in front of the camera, explicitly NOT an additional entropy source. In particular it mitigates the 'RNG not yet warmed up after cold boot' weakness class.

Phase 1 flow:

```

sample 1 (64 B) -> checks -> 2 s hash burst (heats CPU)
sample 2 (64 B) -> checks -> 2 s hash burst
...
sample 5 (64 B) -> checks -> cross-comparison of all 5 samples
  
```

Checks PER sample:

```

diversity >= 32 byte values
monobit 30-70 % ones
  
```

cross-comparison:

```

all 5 samples pairwise distinct
(a stuck RNG is caught instantly)
  
```

Jitter by-product (uncredited, 0 bits): per pause the reached SHA-512 iteration count + exact duration in ns are harvested (scheduler/cache jitter) and appended to the raw data: $5 \times 64 \text{ B} + 4 \times 16 \text{ B} = 384 \text{ B}$ into the hash. Crediting stays a conservative 256 bits — thermal excitation is a quality margin, not a balance item.

Worked example, monobit (per sample, sample run):

```

512 bits read, ones counted: 259
expectation for a fair RNG: 256 (= 512 x 0.5)
standard deviation sigma: sqrt(512 x 0.5 x 0.5) = 11.31
deviation: |259 - 256| = 3 = 0.27 sigma -> PASSED
  
```

(abort region only for gross bias, e.g. 340 ones = 7.4 sigma)

5.4.3 Phase 2 — Radioactive decay: from pulse to bytes

Every detection of a decay product produces a pulse at the P3 VIN tap; the kernel provides a monotonic nanosecond timestamp for it. The complete 8-byte timestamps of all events are hashed — not merely differences or single bits.

Example (real measurement series, ~15 CPM):

event	timestamp t [ns]	delta to predecessor
1	35806374994711	--
2	35807625077266	1.250 s
3	35812116678703	4.492 s
4	35816465826823	4.349 s
5	35820107988436	3.642 s

Byte stream into SHA-512 (big-endian, 8 B per event):

00 00 20 91 5f 8e 3b 57 | 00 00 20 91 aa 03 76 12 | ...

The unpredictability lives in the low-order bytes: even with a known AVERAGE rate, the exact nanosecond instant of each individual decay is quantum-mechanically open.

Only 2 bits per event are credited conservatively although the nanosecond timestamps really carry considerably more — a downward safety margin. For 24 words (256-bit target) at least 128 events are therefore mandatory: 128×2 bits = 256 bits from this source alone.

Statistics guard (audit M4) — worked example with 128 intervals:

```
mean m = 4.0 s -> CPM = 60/m = 15
(1) plausibility window:      1 <= 15 <= 3000          PASSED
(2) coefficient of variation: CV = s/m = 3.8/4.0 = 0.95 ~ 1 PASSED
    (an exponential distribution has CV = 1; 50 Hz hum with jitter
     would give e.g. CV = 0.05 -> IMMEDIATE ABORT)
(3) chi-square vs. exponential distribution, 8 equiprobable bins,
    16 expected each:
    observed: 14 18 15 17 16 13 19 16
    chi2 = (4+4+1+1+0+9+9+0)/16 = 1.75 < 24 (limit)          PASSED
```

5.4.4 Phase 3 — IR camera: frame layout and the 85 % filter

An MLX90640 raw frame (uncalibrated RAM dump):

832 words x 16 bits = 1664 bytes
= 768 pixel ADC values (32 x 24 thermopile)
+ 64 control/calibration words

Example pixel values (hex): fff2 0011 ffe9 002a 0007 ...

The entropy sits in the lowest bits (thermal noise + ADC quantisation) — nevertheless the COMPLETE frame is always hashed.

85 % filter (freshness enforcement) — example:

comparison new frame vs. last ACCEPTED frame:

```
changed words: 731 of 832 -> diff = 87.9 % >= 85 % ACCEPTED
changed words: 615 of 832 -> diff = 73.9 % < 85 % REJECTED
```

The filter forces every credited frame to carry substantially new information (the user moves a hand in front of the optics). A separate guard detects a frozen sensor (200 frames < 5 % difference), and a 5-minute patience window gives the human time without aborting on brief hesitation. Crediting: a conservative 64 bits per accepted frame; minimum 12 frames = 768 bits.

5.4.5 Phase 4 — Dice: base-6 encoding step by step

The roll sequence is encoded as ONE large number in base-6 positional notation (face value minus 1 per digit) and hashed bitwise together with the roll count — lossless and unambiguous:

```
Example with 4 rolls: 4, 1, 5, 2
digits (face-1):      3, 0, 4, 1
value = ((3*6 + 0)*6 + 4)*6 + 1
        = (18*6 + 4)*6 + 1 = 112*6 + 1 = 673
byte encoding: [count as 4 B] [value big-endian]
                = 00 00 00 04 | 02 a1          (0x02a1 = 673)
```

```

Entropy per roll: log2(6) = 2.585 bits
52 rolls -> 52 x 2.585 = 134.4 bits (enforced minimum, > 128!)
100 rolls -> 100 x 2.585 = 258.5 bits (recommendation: full 256-bit
                                         dice coverage in FULL mode)
110 rolls -> 110 x 2.585 = 284.4 bits (LIGHT minimum: + 256 bits HWRNG
                                         = 540 bits >= 2 x 256 with margin)

```

5.4.6 Domain separation: the byte layout of hash inputs

Every source is passed to SHA-512 exactly like this:

```

+-----+-----+-----+-----+
| "BTC-SEEDGEN-v1" | tag   | len(data) 8 B | data   |
| 14 B domain      | e.g. | big-endian    | variable |
|                  | "decay" |              |         |
+-----+-----+-----+-----+

```

Why? Without prefixes a byte stream from source A could be interpreted as source B (format collision). With domain + tag + length every input is globally unambiguous – also versus other programs using SHA-512.

5.4.7 Inside the HMAC combiner

$\text{HMAC-SHA512}(K, m) = H((K' \text{ xor opad}) || H((K' \text{ xor ipad}) || m))$

```

                K = H_dice (64 B)          m = POOL (64 B)
                |                          |
                +-----+-----+          |
                v                          v          v
K' xor ipad --> [ inner hash H( ipad key || POOL ) ]
K' xor opad --> [ outer hash H( opad key || inner ) ] -> FINAL

```

Security property (two-source combiner):

```

unpredictable KEY      + known message      -> FINAL unpredictable
known key              + unpredictable MESSAGE -> likewise
-> An attacker would need to control dice AND all hardware sources.

```

This is exactly why plain XOR would be the weaker choice: XOR is a combiner too, but fragile if one source could adaptively observe the other; the hash/HMAC construction destroys any structure of the inputs and is robust against such dependencies.

5.4.8 BIP39 encoding: fully worked example

For verifiability, here is the official BIP39 test vector with 128-bit all-zero entropy (in a real run ENTROPY = FINAL[:32] takes this place; the arithmetic is identical, for 24 words with 256+8 bits and an 8-bit checksum):

Step 1 – entropy:

ENT (128 bits) = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Step 2 – checksum:

SHA-256(ENT) = 374708fff7719dd5979ec875d56cd228...

first byte = 0x37 = 00110111b

CS = first ENT/32 = 4 bits = 0011b = 3

Step 3 – concatenation ENT || CS (132 bits), split into 12 x 11 bits:

[000000000000] [000000000000] ... (11 groups) ... [000000000011]

indices: 0 0 0 0 0 0 0 0 0 0 0 3

Step 4 – wordlist (line = index + 1):

index 0 -> abandon (x11), index 3 -> about

Mnemonic: abandon abandon abandon abandon abandon abandon

abandon abandon abandon abandon abandon about

= official BIP39 test vector -> arithmetic verified

The final 11-bit block carries the 4 checksum bits as its low bits — which is why the zero vector does not end in 'abandon' but in 'about' (index 3). Every BIP39 wallet worldwide performs exactly this computation and detects typos via the checksum (for 24 words, 8 of 264 bits are checked; gross errors are additionally caught by wordlist lookup).

5.4.9 Wallet side: from word sequence to addresses

```
Mnemonic (+ optional passphrase)
| PBKDF2-HMAC-SHA512, 2048 iterations, salt = "mnemonic"+pass
v
512-bit wallet seed
| BIP32: HMAC-SHA512("Bitcoin seed", seed) -> master key + chaincode
v
m / 84' / 0' / 0' / 0 / i      (BIP84, native SegWit)
| | | | | +- address index i = 0,1,2, ...
| | | | | +--- external (0) / change (1)
| | | | | +----- account 0
| | | | | +----- coin: Bitcoin (0)
| | | | | +----- purpose: BIP84
v
private keys -> public keys (secp256k1) -> bc1q... addresses

Import reference Electrum (offline): Options -> BIP39, path m/84'/0'/0'
```

6. Security Mechanisms of the Python Script

Before the run

- Security-rules box with mandatory confirmation (offline rules, smart-TV ban, no phones/cameras/third parties in the room)
- Root check with clear sudo guidance; abort under python3 -O (self-tests would run incompletely)
- Remote/recording detection: SSH, tmux/screen, script(1), VNC/xrdp, web terminals (ttyd, wetty, cockpit et al.) and pty recorders — sudo-robust via the /proc process ancestry, not just environment variables; plus a mandatory stdout TTY (pipe/tee/redirect would store the seed persistently → abort)
- Enforced radio shutdown: rfkill immediately + dtoverlay entries (firmware level, from boot) — with verification
- Mandatory Ethernet cable check: link detection, unplug demand, technical re-measurement of the confirmation
- Swap disabled plus RAM hardening: mlockall() pins all memory pages in RAM (effective even if swapon fails), PR_SET_DUMPABLE=0 blocks ptrace//proc reads at runtime, core dumps disabled (RLIMIT_CORE=0), protection against interactive interpreter mode (-i/PYTHONINSPECT)
- Cryptographic self-test before every run: official BIP39 vectors, a known-answer vector of the complete derivation chain (pinned FINAL + indices) and fault-injection probes; wordlist SHA-256 verification; in automatic mode CLI parameters are validated before any system-changing step
- Hardware health check: 10 live signals per source with display, incl. measurement of the real count rate for honest duration estimates

During generation

- Enforced minima: ≥ 128 decay events (24 words), ≥ 12 camera frames, ≥ 52 dice rolls; entropy balance $\geq 2\times$ target
- Statistical decay test: CPM plausibility window, coefficient of variation and chi-square against the exponential distribution — with dead-time correction (the 30 ms debounce is subtracted, otherwise false alarms at high rates) and a rate-adaptive timeout (slow tubes are not falsely aborted); detects periodic interference that would fool simple checks
- Camera freshness filter: only frames with $\geq 85\%$ pixel change; separate frozen-sensor guard; 5-minute patience window for the human
- Dice health checks: minimum diversity, bias detection and a chi-square test over the six faces (loaded dice and systematic typos are caught)
- Geiger input: debouncing (30 ms), automatic edge selection and hardware-verified bias-free GPIO (pull none — the high-impedance P3-VIN tap with 10 k Ω in series is choked by any internal pull); adjustable via --gpio-pull, with a 60-second auto-fallback across all pull modes and a diagnostic line (chip, pull, idle level, edge) at start

At display and completion

- Final offline check immediately before seed display: WLAN, Bluetooth and Ethernet are re-measured; on any interim (re-)activation a security abort with tamper warning — the seed never appears
- Mock mode: randomly masked word indices instead of words, and all transparency secrets (H_dice, FINAL, ENTROPY, dice sequence) masked too — test seeds are unusable; a mock run demonstrably never mutates the host (no rfkill, no swapon, no config.txt entries)
- Paper verification: red instructions (write words WITH their numbers — order!), re-entry ONLY from paper, tolerant parser, exact match with position diagnostics
- Trace-free finish: screen and scrollbar erase in terminal-safe order (home → 2J → 3J, with clear fallback and repetition), optional bash-history cleanup
- No secret is ever written to storage: all secrets exist exclusively in RAM. The only disk writes are the announced hardening steps (config.txt overlays with backup and fsync, optional systemd unit, history cleanup); emergency screen wipe also on SIGTERM/SIGHUP/SIGQUIT after seed display

Dual-computation validation against bit flips (since v1.1)

The Raspberry Pi 5 has no ECC RAM — a single flipped bit between computation and display would silently put a wrong seed on paper. Against this the script computes the complete derivation chain (source digests → POOL → HMAC-SHA512 → FINAL → entropy truncation → BIP39 word coding) twice, fully and independently, from the same inputs (the OS nonce is sampled exactly once for this). Only when FINAL, indices and finished word sequences of both runs match does it proceed. In addition: a BIP39 roundtrip (words → entropy → checksum → re-encoding → comparison) and a wordlist re-verification right before output — the in-memory list is serialized identically to the load format and checked against the SHA-256 reference hash again, because the roundtrip alone would be blind to a table corrupted in RAM. Any mismatch aborts loudly with its own exit code 4 and without any partial display.

LIGHT mode

For setups without the Geiger counter and camera, LIGHT mode derives seeds from HWRNG + dice. The HMAC combiner carries the security burden; the enforced minimum is 110 rolls ($110 \times \log_2 6 + 256$ bits HWRNG = 540 bits $\geq 2 \times 256$ with margin). A mode tag in the pool computation cryptographically separates LIGHT and FULL derivations (domain separation) — identical inputs demonstrably yield different seeds in the two modes.

Low-count-rate CPM warning dialog

If the count rate measured in the health check is below 4 CPM, a dialog explains adjusting the threshold potentiometer on the CAJOE board (2–4 turns counter-clockwise, with a high-voltage warning) and offers to repeat or knowingly continue — instead of a silent, hours-long run.

7. Theory: Quantum-Computer Attack — Hardware-Wallet Seed vs. This 24-Word Seed

A future fault-tolerant quantum computer threatens Bitcoin along two separate avenues that must be kept clearly apart:

Avenue A — Shor's algorithm against the elliptic-curve cryptography (secp256k1): once an address's public key is known (at the latest on first spend, permanently for reused addresses), Shor could derive the private key in polynomial time. Honest statement: NO seed generator in the world protects against this avenue — neither a hardware wallet nor this system — because it attacks the signature curve, not the seed entropy. The defence lies at the protocol level (future post-quantum signatures) and in practice: never reuse addresses, migrate funds to new schemes once real quantum capability emerges.

Avenue B — Grover's algorithm against the seed entropy (brute force): Grover speeds up unstructured search quadratically — the effective security of n -bit entropy drops to $n/2$ bits. This is precisely where the difference shows:

Scenario	classical security	quantum security (Grover)	assessment
24 words, full 256 bits (this system, evidenced)	2^{256}	2^{128}	permanently secure — 2^{128} is considered out of reach even for quantum machines
24 words from an intact hardware wallet	2^{256} (vendor claim)	2^{128}	equivalent — PROVIDED the RNG truly delivered in full
24 words from a flawed/backdoored RNG (documented real-world cases: reduced effective entropy, e.g. 64–128 bits)	2^{64} – 2^{128}	2^{32} – 2^{64}	at risk to trivially breakable — and the user cannot tell
12 words (128 bits)	2^{128}	2^{64}	classically fine, borderline in the quantum era — hence 24 words recommended here

Conclusion: this system's advantage in the quantum context is not some magical quantum resistance of the signatures (nobody has that), but the GUARANTEE of the full 256 bits of input entropy. Grover halves exponents — whoever starts with evidenced 256 bits keeps a 128-bit quantum margin; whoever unknowingly started from a weakened RNG may fall into breakable territory under Grover. The multi-source architecture with transparency mode and an enforced balance makes precisely this silent weakening impossible. Additionally: while an address is unused (public key known only as a hash), the hash barrier also shields against avenue A for the time being — one more reason for address hygiene.

8. Operating Procedure (Quick Reference)

- Setup (once, online permitted): `sudo python3 install_seedgen.py` → `sudo reboot`
- Start: `sudo python3 btc_seedgen.py` → choose language → confirm security rules
- Automatic: radio shutdown, Ethernet unplug (enforced), swap off, health check with success jingle
- Configuration: mode (standard/transparency), word count, events/frames/rolls (minima enforced; realistic duration from measured count rate)
- Phases 1–4: HWRNG → decay → camera (move your hand!) → enter dice
- Final offline check → seed display → words WITH numbers on paper → verification FROM PAPER → Enter erases screen+scrollback → close terminal
- Recommendation for real use (FULL): 24 words, ≥ 512 decay events, ≥ 100 rolls; in LIGHT mode the enforced minimum is 110 rolls. Import verification on a second offline device (Electrum: Options → BIP39; path `m/84'/0'/0'`)
- Hardware diagnostics (tools/): `cajoe_test.py` measures the count rate, `gpio_decay_diagnose.py` finds the right pull/edge combination (tests all nine, changes nothing on the system), `camera_test.py` and `camera_raw_test.py` check the MLX90640; below 4 CPM the main script itself walks through the potentiometer adjustment

Script exit codes:

Code	Meaning
0	success — seed generated and run completed cleanly
2	security or configuration abort (source failed, statistics test, parameter error, user declined a gate)
3	final offline check violated — radio/Ethernet (re-)activated meanwhile, the seed was never displayed
4	dual-computation validation failed — suspected transient memory error (bit flip), repeat the run
130	aborted by the user (Ctrl+C)

9. Limitations and Residual Risks (an honest view)

- The Shor risk to the signature curve remains (see ch. 7) — solvable at protocol level, not by any generator
- Python cannot guarantee RAM overwriting (immutable objects); mitigation is via swap/core-dump lockdown and process exit — for maximum paranoia: power-cycle the Pi after the run
- Health checks detect defects, not perfectly disguised chip manipulation — the architecture (dice HMAC) counters that, not statistics
- Physical security of the paper backup lies outside the system (metal plate, safe storage, optionally a passphrase as the 25th word in the wallet)
- The monitor is part of the trust chain (hence the smart-TV ban in the rules)
- A single run does not replace operational discipline: test runs with throwaway seeds before real use remain mandatory
- Limits of the dual run: deterministic program bugs hit both runs identically (the known-answer vectors in the self-test stand against that); bit flips in the raw data BEFORE both runs are harmless (corrupted randomness is still randomness); a flip only in the display buffer is caught by the BIP39 checksum during the test restore — which is why import verification before the first deposit remains mandatory
- pty recorders are not fully detectable in principle (e.g. asciinema running as a python process) — detection covers the common tools but does not replace the rule of performing the ceremony only at a freshly started local console
- mlockall/PR_SET_DUMPABLE do not protect against root processes, cold-boot or DMA attacks — the core recommendation stands: one fresh boot per seed ceremony, then power off

10. Integrity and Reproducibility

The repository ships a closed checksum chain: sha256.txt covers all project files (scripts, documentation in docs/, tools in tools/) — after cloning or any transfer to the Pi: `sha256sum -c sha256.txt`, then `sudo python3 btc_seedgen.py --selftest`. The self-test verifies the official BIP39 vectors, the wordlist, the dual-computation logic via fault injection and a known-answer vector of the complete derivation chain (pinned FINAL and pinned indices) — a deterministic combiner bug would surface here before any seed is ever created. In transparency mode every intermediate value (H_hw, H_dec, H_cam, H_os, H_dice, POOL, FINAL, ENTROPY) is displayed and independently recomputable with standard tools — at no point does the system demand trust in unsubstantiated claims.

11. Glossary

Term	Meaning
BIP39	standard for encoding entropy as a memorable word sequence incl. checksum
Entropy	measure of unpredictability; conservatively accounted in bits here
HMAC	keyed hash construction; used here as a robust two-source combiner
Domain separation	unambiguous tagging of hash inputs, prevents format confusion
CPM	counts per minute — Geiger counter rate
Poisson process	statistics of independent random events; hallmark of genuine decay ($CV \approx 1$, exponential intervals)
Grover/Shor	quantum algorithms: quadratic search speed-up resp. breaking public-key cryptography
Air gap	physical separation of a system from all networks
KAT	known-answer test: pinned expected result against which the self-test checks the derivation chain
Dual computation	two independent executions of the derivation chain compared before display (bit-flip detection)

12. License and Disclaimer

This project (source code, installer and this documentation) is released under the MIT License — Copyright (c) 2026 Quantum Seed Generator Project. Anyone may use, audit, modify and redistribute the software free of charge; the sole condition is retaining the license and copyright notice. The full license text resides in the LICENSE file of the repository. The software is provided 'AS IS', without warranty of any kind.

This system is a DIY project for technically proficient users. Responsibility for operation, backup custody and the handling of cryptocurrencies rests entirely with the user. Before use with real funds, personal test runs and import verification on an independent offline device are mandatory.